

Bakoda

Otel Oda Rezervasyonu için Uçtan Uca Test ve Dağıtım Pipeline'ı

MTH2526-B25 — Bulut Mimarilerinde Test Mühendisliği

Marmara Üniversitesi · 2025–2026 Bahar

Grup: Arif Batuhan Bahar · Burak Tuzcu

Neden bir test pipeline'ı?

PROBLEM

- Otel rezervasyonunda müsaitlik, çakışma ve onay belgesi kritiktir.
- Aynı oda/tarih çift-rezervasyonu, hatalı durum geçişleri ve sessiz veri kaybı production'da pahalıdır.
- Manuel test ölçeklenmez → otomasyon ve gözlemlenebilirlik şarttır.

ÇÖZÜM — Bakoda (FastAPI)

- Otel/oda ara, müsaitlik kontrol et, rezervasyon oluştur · güncelle · iptal et.
- Onay belgesi LocalStack S3'e yazılır; auth, favoriler, ödeme ve iletişim uçları kalıcıdır.
- Basit React UI üzerinden tam test / deploy / monitoring yığını.

Tüm bileşenler tek bakışta

API

FastAPI + Uvicorn · 34 uç
/api · /health · /metrics · /docs

Veritabanı

PostgreSQL 16 · SQLAlchemy 2
Alembic (5 migration)

Object Storage

LocalStack S3 + boto3
bakoda-confirmations bucket

Kubernetes

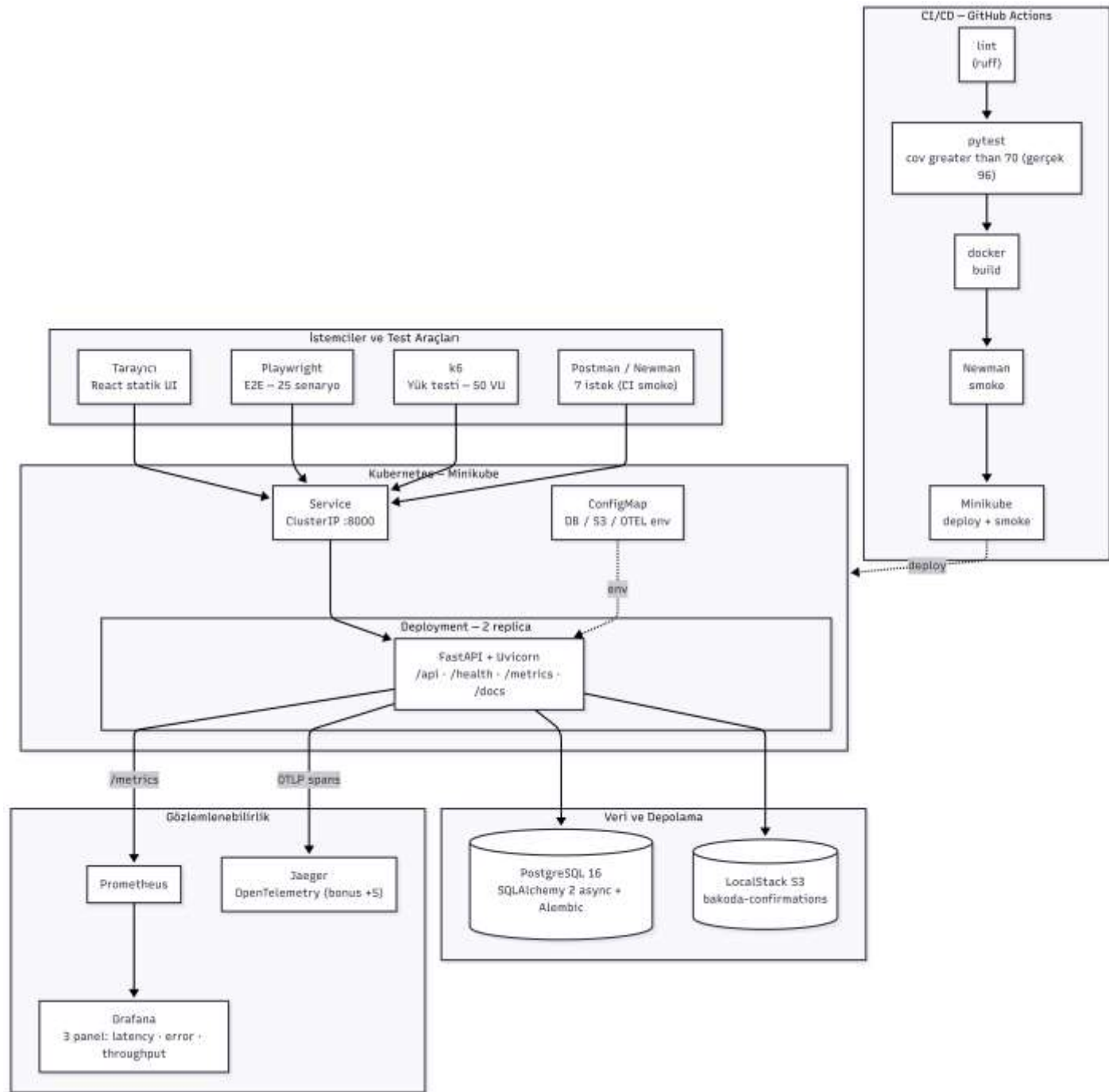
Minikube · 2 replica
Deployment + Service + ConfigMap

Monitoring

Prometheus → Grafana
3 panel + /metrics scrape

Tracing (bonus +5)

OpenTelemetry → Jaeger
FastAPI + SQLAlchemy span'leri



Test piramidi: 306 test fonksiyonu

E2E 25

Playwright · API + UI akışları (CI'da)

Integration 145

pytest-asyncio + httpx + Testcontainers PG

Unit 136

servisler · yardımcılar · şema · factory

COVERAGE

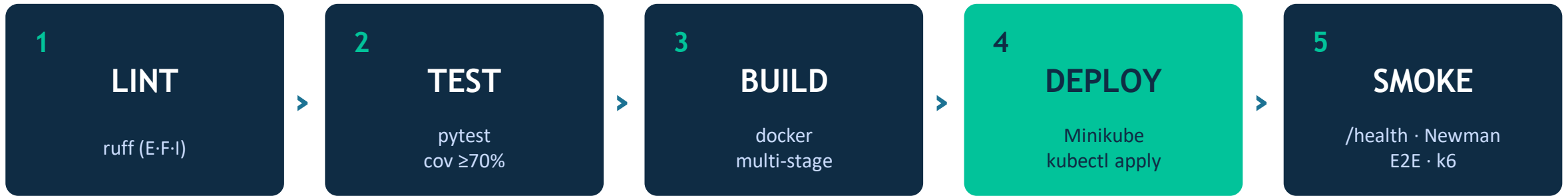
%98.5

hedef $\geq$ %70
(fail_under = 70)

293 test geçer · 0 hata

Factory Boy + Faker (Room/Booking/Hotel/User) suite genelinde · Postman 7 istek → Newman · öne çıkan: 409
çakışma, iptal sonrası müsaitlik, ownership, kalıcılık.

GitHub Actions: lint → test → build → deploy → smoke



Deploy stratejisi: Minikube'de gerçek k8s deploy — image yüklenir, manifestler uygulanır, kubectl rollout status beklenir, ardından servise /health smoke. 2 replica ile rolling update; readiness geçmeden trafik yok. Rollback: kubectl rollout undo.

Tüm test piramidi otomatik: 293 unit+integration + 25 E2E (Playwright) + k6 p95<500 regresyon kapısı · ruff lint temiz.

Prometheus + Grafana — 3 panel

01

Request Latency

p50 / p95 / p99

histogram_quantile üzerinden gecikme dağılımı

02

Error Rate

5xx / toplam

hata oranı zaman serisi (yük altında %0)

03

Throughput

req/s (handler)

uç bazında istek hızı

Bonus — OpenTelemetry → Jaeger: FastAPI ve SQLAlchemy otomatik enstrümantasyonu; OTLP gRPC ile span'ler Jaeger UI'da (:16686). Bir config bayrağı ile açılıp kapanır (testte kapalı). Latency spike'ında ilgili SQL/booking span'ine inilir.

Projeyi anlatan rakamlar

%98.5

test coverage (hedef ≥ 70)

293

unit + integration testi · 0 hata

306

toplam test fonksiyonu

465 ms

k6 p95 (<500, CI kapısı)

34

REST API ucu

25

Playwright E2E (CI'da)

3

Grafana paneli

+5

OpenTelemetry bonus

En hatırdaki kalan 3'er madde

ÖĞRENİLEN DERSLER

- Testcontainers ile gerçek PostgreSQL — mock'tan daha güvenilir integration testi.
- Factory Boy + Faker — gerçekçi, bakımı kolay test verisi suite genelinde.
- Metrik + trace birlikte — latency spike'ında doğrudan span'e inmek.

ZORLUKLAR

- Async SQLAlchemy + fixture yaşam döngüsü ile test izolasyonu.
- Test gürültüsü/dayanıklılık: OTel'i test'te kapatmak, LocalStack'i Docker yoksa skip etmek.
- CI'da gerçek Minikube deploy + E2E/k6 otomasyonu (canlı stack + seed).

Teşekkürler — Sorular?